PowCoin: Student Manual

PowCoin is a small proof-of-work cryptocurrency built for studying how full nodes, wallets, transactions, mining, and peer synchronization fit together. This manual uses a new, independent network on your own computer. PowCoin is experimental software; do not use it for real money or expose its API to the public Internet.

Contents

1. What the project does
2. Install and run a local node
3. First practical exercise: mine and transfer coins
4. Project files and architecture
5. Network configuration and identity
6. Wallets, addresses, and signatures
7. Transactions, balances, and confirmations
8. Blocks, proof of work, and rewards
9. Full-node validation and persistent data
10. Peers and synchronization
11. Command-line and desktop interfaces
12. HTTP API and browser explorer
13. Security boundaries and safe handling
14. Testing, troubleshooting, and study questions

1. What the project does

A **wallet** creates an Ed25519 key pair and signs transfers locally. A **full node** checks every block and transaction against the same consensus rules, stores the accepted chain in SQLite, and serves an HTTP API. A **miner** asks a node for a candidate block, searches for a valid nonce, and submits the completed block. **Peers** exchange blocks and pending transactions so their nodes can converge on the same valid chain.

The project uses an account model: the node derives each address's spendable balance and next transaction nonce by replaying the chain. It is not a Bitcoin implementation and has no UTXO set. One coin contains 100,000,000 integer base units; the CLI accepts decimal coin amounts with up to eight digits after the decimal point.

The browser explorer shows chain and account information. The optional Tkinter desktop app creates or opens wallets, sends signed transfers, and mines blocks. The node never needs a wallet's private key to validate a transfer or pay a mining reward.

2. Install and run a local node

Requirements

- Python 3.10 or newer; Windows examples below use the `py` launcher.
- The project's `cryptography` dependency, installed by `pip install -e .`.
- Tkinter only if you want the desktop app. The CLI and node work without it.

Extract the student package and open PowerShell in the inner `powcoin` directory, the one containing `pyproject.toml`. Install into a fresh virtual environment:

```
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -e .
.\.venv\Scripts\python.exe -m powcoin --help
```

For Linux or macOS, use `python3 -m venv .venv`, `.venv/bin/python -m pip install -e .`, and replace the Windows Python path in later commands with `.venv/bin/python`.

Create a **new** configuration for this exercise. A lower proof-of-work difficulty makes classroom mining faster:

```
.\.venv\Scripts\python.exe -m powcoin init --out network.json --name StudentCoin --symbol STC --
pow-bits 12
```

The command creates `network.json` and displays its chain ID. Do not edit this file after mining begins. Start one node in the current terminal:

```
.\.venv\Scripts\python.exe -m powcoin node --config network.json --data data/student --host
127.0.0.1 --port 8844
```

Leave that terminal open. In a second PowerShell terminal, from the same `powcoin` directory, check the node and open the browser explorer:

```
.\.venv\Scripts\python.exe -m powcoin status --url http://127.0.0.1:8844
```

Open `http://127.0.0.1:8844/` in your browser. The initial height is 0: the deterministic genesis record. Press `Ctrl+C` in the node terminal to stop it. Restart with the **same** `network.json` and `data/student` directory to keep the chain.

What these files mean

`network.json` holds consensus settings and defines chain identity. `data/student/chain.sqlite3` holds accepted blocks and pending transactions; SQLite may also create `-wal` and `-shm` files. `data/student/node.lock` prevents two live processes from using the same data directory. A different experiment should use a different configuration file **and** data directory.

3. First practical exercise: mine and transfer coins

Keep the local node running. In the second PowerShell terminal, create two wallets. The CLI prompts for each password and asks you to repeat it:

```
New-Item -ItemType Directory -Force wallets | Out-Null
.\.venv\Scripts\python.exe -m powcoin wallet-create --wallet wallets/alice.pem
.\.venv\Scripts\python.exe -m powcoin wallet-create --wallet wallets/bob.pem
$alice = (Get-Content wallets/alice-address.txt -Raw).Trim()
$bob = (Get-Content wallets/bob-address.txt -Raw).Trim()
```

The address files are public information. The `.pem` files and their `-password.txt` sidecars are private. The sidecars contain the wallet passwords in plaintext, even though the PEM keys themselves are encrypted.

Mine three blocks to Alice. With the default reward maturity of two blocks, the reward from height 1 becomes spendable at height 3. A block can take an unpredictable amount of time to find; `--pow-bits 12` makes this exercise practical on an ordinary computer.

```
.\.venv\Scripts\python.exe -m powcoin mine --config network.json --address $alice --blocks 3 --
url http://127.0.0.1:8844
.\.venv\Scripts\python.exe -m powcoin balance --address $alice --url http://127.0.0.1:8844
```

Send one coin from Alice to Bob. The CLI loads Alice's encrypted PEM and asks for its password. The amount and fee are in **coins**; the signed transaction carries integer base units.

```
.\.venv\Scripts\python.exe -m powcoin send --config network.json --wallet wallets/alice.pem --to
$bob --amount 1 --fee 0.001 --url http://127.0.0.1:8844
.\.venv\Scripts\python.exe -m powcoin mine --config network.json --address $alice --blocks 1 --
url http://127.0.0.1:8844
.\.venv\Scripts\python.exe -m powcoin balance --address $bob --url http://127.0.0.1:8844
```

The `send` command prints a transaction ID. Before mining, the transaction is **pending** in the mempool. Once a block includes it, the transaction becomes **confirmed**. Use `powcoin tx --id TRANSACTION_ID --url http://127.0.0.1:8844` to see its status and current-chain confirmation count. A reorganization can change that count, so a confirmation is not an irrevocable promise.

4. Project files and architecture

File or directory	Responsibility
`pyproject.toml`	Python package metadata, dependency, and CLI entry points.

File or directory	Responsibility
`powcoin/consensus.py`	Canonical encoding, hashes, signatures, block rules, proof of work, and state transitions.
`powcoin/wallet.py`	Encrypted Ed25519 keys, public addresses, and local signing.
`powcoin/ledger.py`	Chain, balances, pending pool, SQLite persistence, and reorganizations.
`powcoin/network.py`	HTTP server and client, peer sync, token checks, and optional continuous mining.
`powcoin/cli.py` and `__main__.py`	`python -m powcoin` commands and argument parsing.
`powcoin/desktop.py`	Tkinter wallet and mining app.
`powcoin/dashboard.py`	Browser explorer page served by the node.
`powcoin/loopback_dashboard.py`	Optional read-only proxy for a token-protected node.
`tests/`	Consensus, wallet, desktop, network, and discovery tests.

The normal flow is: wallet signs a transfer; CLI or desktop sends the signed object to the node; the node verifies and queues it; a miner builds and solves a candidate block; the node validates that block, updates balances, persists it, and shares it with peers. The private key stays with the wallet.

5. Network configuration and identity

`powcoin init` creates a JSON object with these fields. Every node on one network must use the **same values**.

Field	Meaning
`version`	Configuration format version; currently 1.
`name`, `symbol`	Human-readable currency labels.
`network`	Random network discriminator generated by `init` unless supplied explicitly.
`pow_bits`	Fixed proof-of-work difficulty from 8 to 28.
`reward`	Initial block subsidy in integer base units.
`halving_interval`	Number of block heights between subsidy halvings.
`maturity`	Number of heights before a mining reward can be spent.
`genesis_time`	Unix time anchor used in block timestamp checks.

The **chain ID** is SHA-256 of validated configuration serialized as canonical JSON. Changing any value creates a different chain identity, even if the names look similar. Genesis is a deterministic, unmined record with height 0 and no coins. Nodes reject saved data whose chain ID differs from the supplied configuration. CLI transfers and mining also check that the remote node's chain ID matches their local configuration.

Canonical JSON sorts keys, removes unnecessary whitespace, escapes non-ASCII characters, and rejects NaN. This makes hashing independent of ordinary formatting choices such as indentation or key order. Keep one shared configuration file when experimenting with more than one node.

6. Wallets, addresses, and signatures

A wallet generates a random Ed25519 private key. The corresponding raw public key is 32 bytes. The address is the lowercase hexadecimal SHA-256 hash of those public-key bytes. The address and public key can be shared; the private key and its password cannot.

Wallet creation writes three files: `NAME.pem` (encrypted PKCS#8 private key), `NAME-password.txt` (plaintext password), and `NAME-address.txt` (public address). The password must contain at least 10 characters. Wallet creation refuses to overwrite an existing bundle. Keep a private backup of the PEM and password together; losing either can make the wallet unusable. Do not commit these files to Git or send them to classmates.

`wallet-address --wallet PATH` decrypts a wallet and displays its public address. A transaction is signed over every field except the signature field. Nodes verify the signature with the included public key and check that the public key hashes to the claimed sender address. A node does not store or receive the private key or the wallet password.

7. Transactions, balances, and confirmations

Each signed transaction has exactly these fields: `chain`, `sender`, `public_key`, `recipient`, `amount`, `fee`, `nonce`, and `signature`. The transaction ID is the SHA-256 hash of the complete signed canonical JSON. All amount and fee fields inside the transaction are integer base units.

A node checks the chain ID, address format, sender/public-key match, Ed25519 signature, positive amount, nonnegative fee, sender's next nonce, and enough spendable balance for **amount plus fee**. The sender's nonce starts at 1 and increases by one with each accepted transfer; this prevents replay and orders transfers from one address. Sending to yourself is permitted but still consumes a nonce and fee.

The **confirmed balance** comes from the accepted chain. The **immature balance** is earned mining reward that has not reached its unlock height. `next_block_available` is a preview after rewards due at the next height mature and pending transfers are applied. `next_nonce` accounts for pending transfers. A transaction in the mempool has zero confirmations; a transaction included in the current chain has at least one. The confirmation count can change if a stronger branch replaces the current chain.

The mempool holds up to 1,000 pending transactions. A candidate block takes at most the first 100. A transaction that fails validation is rejected rather than silently changing balances. If a block makes pending transactions invalid, the node rebuilds its pool against the new state. The node also tries to requeue still-valid transactions from an orphaned branch after a reorganization.

8. Blocks, proof of work, and rewards

A block has a header containing version, chain ID, height, previous hash, timestamp, nonce, miner address, reward, and Merkle root. The full block also contains transactions and its header hash. The block hash is SHA-256 of canonical header JSON. Mining changes the nonce until the hash, interpreted as an integer, is at or below the target `2^(256 - pow_bits) - 1`.

The Merkle root commits to all signed transactions. Leaves hash the complete signed transaction; pairs are hashed together. An odd last leaf is duplicated. An empty block uses SHA-256 of empty bytes as its Merkle root. A valid block has at most 100 transactions and at most 128,000 canonical JSON bytes.

Each block reward must equal the subsidy for its height **plus the fees** in that block. The subsidy halves after each configured interval. The reward is locked until `block height + maturity`. At a new height, rewards due at that height mature first, then transactions run in order. A transfer later in a block may spend a receipt from an earlier transfer in that block; the new block's mining reward cannot fund a transfer in its own block.

Nodes check the previous hash, exact next height, chain ID, timestamp, Merkle root, block hash, proof-of-work target, transaction order, balances, nonces, and reward. Timestamps cannot move backward or exceed the local clock by more than 120 seconds. Proof of work is fixed difficulty in this prototype; it does not retarget as the network changes.

9. Full-node validation and persistent data

The full node does not accept a peer's claim that a chain is valid or that it has more work. It downloads candidate blocks, verifies them from genesis, derives balances and nonces itself, and adopts a chain only when its total validated work is **strictly greater**. With this fixed difficulty, more blocks means more work. Equal-work ties keep the local tip until one branch grows stronger.

SQLite stores blocks, pending transactions, and chain identity. Write-ahead logging and synchronous commits help protect the local database. On restart, the node replays and validates saved blocks. A lock prevents two nodes from using one data directory at the same time. If you need a reliable backup, stop the node and copy the whole data directory, including any SQLite WAL files. Copying only a live `chain.sqlite3` file may omit recent transactions.

A node validates a received block before appending it. If a longer valid branch is adopted, it rebuilds balances, nonces, and the pending pool from that branch. Historical side branches are not retained as a separate archive. Never delete a data directory merely to solve an error without first understanding what chain history it holds.

10. Peers and synchronization

Peers are configured with one or more `--peer URL` arguments. A node periodically asks each peer for status, fetches blocks when needed, checks the chain itself, and exchanges pending transactions. A node can run alone on `127.0.0.1`; peer networking is optional for the first exercises.

For a safe same-computer experiment, stop the first node and restart it with a shared token. Create the token file without displaying it:

```
New-Item -ItemType Directory -Force .deploy | Out-Null
.\.venv\Scripts\python.exe -m powcoin token | Set-Content -NoNewline .deploy/shared_token
$env:POWCOIN_TOKEN = (Get-Content .deploy/shared_token -Raw).Trim()
.\.venv\Scripts\python.exe -m powcoin node --config network.json --data data/student --host
127.0.0.1 --port 8844
```

In a second terminal, load the same token and start a second node with a **different** data directory and port:

```
$env:POWCOIN_TOKEN = (Get-Content .deploy/shared_token -Raw).Trim()
.\.venv\Scripts\python.exe -m powcoin node --config network.json --data data/peer2 --host
127.0.0.1 --port 8845 --peer http://127.0.0.1:8844
```

Compare `status` on ports 8844 and 8845 after mining on the first node. The second node should learn accepted blocks. Both nodes must use the same `network.json` and token, but each needs its own data directory and listener port. Non-loopback binds require a token; do not expose a classroom node to the public Internet. The built-in server uses HTTP, so a shared token is not protected by transport encryption on an untrusted network.

The code also includes a specialized VPN peer-discovery mechanism. It validates token-based challenge proofs before sending authenticated requests to discovered peers, and limits accepted origins to a configured private subnet. That feature is for a deliberately configured private network; the localhost exercise above uses explicit peers and does not require discovery.

11. Command-line and desktop interfaces

The main command is `python -m powcoin`. Important subcommands are `init`, `token`, `wallet-create`, `wallet-address`, `node`, `status`, `balance`, `send`, `mine`, and `tx`. Run a subcommand with `--help` to see its flags. `node --mine-to ADDRESS` enables continuous background mining; `mine --address ADDRESS --blocks N` mines a finite number of blocks from the CLI. Both pay rewards to a **public address**, so the miner does not need the wallet's private key.

The optional desktop app is started with `python -m powcoin.desktop`. It expects `network.json`, a reachable full node, and a token file or `POWCOIN_TOKEN` environment variable. Use the token-protected node from Section 10, then run:

```
.\.venv\Scripts\python.exe -m powcoin.desktop --config network.json --node-url
http://127.0.0.1:8844 --token-file .deploy/shared_token
```

The app can create or open an encrypted wallet, display balances, prepare and confirm a signed transfer, and mine to a chosen reward address. Opening a wallet decrypts its key for the current app session; closing the app ends that in-memory session. The receiving wallet need not be open. The app checks the node's chain ID against `network.json`, so it will refuse a node from a different network. Tkinter must be available in your Python installation.

12. HTTP API and browser explorer

The full node serves a browser explorer at `/` and a JSON API. With a token configured, requests except the discovery challenge route require `Authorization: Bearer TOKEN`. Without a token, a node bound only to localhost is unauthenticated. The CLI sends the token automatically when `POWCOIN_TOKEN` is set.

Method and route	Purpose
`GET /status`	Chain ID, height, tip, work, pending count, mining and peer status.
`GET /config`	Consensus configuration.
`GET /blocks?start=0&count=50`	Page of canonical blocks, including genesis at height 0.
`GET /accounts/ADDRESS`	Confirmed and immature balance, pending preview, and nonces.
`GET /mempool` or `/mempool/details`	Pending signed transactions, optionally with IDs.
`GET /transactions/TXID`	Pending, confirmed, or unknown status.
`GET /template?address=ADDRESS`	Candidate block for a miner.
`POST /transactions`	Submit a complete signed transaction.
`POST /blocks`	Submit a complete mined block.

The peer-discovery routes are `POST /announce` and `GET /discovery-proof?nonce=...`; they are for the specialized private-network discovery protocol. There are no HTTP endpoints to unlock wallets, import private keys, or administer the computer. The optional `loopback_dashboard.py` is a separate **read-only** proxy for a protected node; it does not expose the full API and cannot be used as the transfer or mining endpoint.

13. Security boundaries and safe handling

PowCoin is a teaching prototype, not an audited financial system. The most important boundary is between **public code and chain data** and **private credentials**. Keep wallet PEM files, plaintext password sidecars, and shared API tokens out of GitHub, ZIP/RAR releases, screenshots, and chat messages. The project's `.gitignore` helps when using Git, but it does not remove files from an archive or erase files already committed to Git history.

The shared API token proves membership in a group; it does not identify individual students or encrypt traffic. The built-in HTTP listener has no TLS. Discovery proofs prevent a discovered peer from receiving the token before proof, but explicit configured peers are trusted destinations. The implementation has fixed difficulty, bounded resource limits, and no production-grade rate limiting, public peer reputation, or monetary safeguards. Run exercises on localhost or a deliberately isolated trusted network.

If a private key or token is accidentally published, deleting the current file is insufficient: copies may remain in downloads or Git history. Treat the credential as exposed, replace the token across every node that uses it, and move any valuable wallet balance to a newly generated wallet where possible. For class exercises, the

easiest safe practice is to generate fresh disposable credentials in each student's own copy.

14. Testing, troubleshooting, and study questions

Run the project tests from the `powcoin` directory:

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

The tests exercise consensus rules, wallet and desktop workflows, HTTP networking, and discovery behavior. They use temporary state; a passing suite does not make the software production safe.

Symptom	Likely cause and next check
`ModuleNotFoundError`	Use the virtual environment's Python and run `pip install -e .` from the directory with `pyproject.toml`.
`network.json` is missing	Run `powcoin init` once for a new exercise network. Do not overwrite an existing config with chain data.
`Configuration differs from the saved chain`	The data directory belongs to another chain ID; restore its original config or choose a new data directory.
`Data directory already used`	Another node is using that directory. Stop it or choose a separate directory.
`HTTP 401`	The client and node have different or missing tokens. Load the correct `POWCOIN_TOKEN`.
`insufficient spendable balance`	A reward may still be immature, or a pending transfer has reserved funds. Check `balance`.
`wrong nonce or replay`	The sender already used that nonce or pending transactions changed the next nonce. Refresh account state.
GUI cannot start	Confirm Tkinter, the node URL, matching `network.json`, and a readable token file.
Peers disagree on chain	Compare chain IDs, tips, validated work, connectivity, and whether a tie has been broken by another block.

Study questions: Why must every node independently validate a peer's blocks? Why does a signature need the sender's nonce and chain ID? What changes if reward maturity is zero? Why is the password sidecar sensitive even though the PEM is encrypted? Why does `gitignore` not make a whole-folder archive safe to publish? What can happen when two valid branches have equal work?

This manual describes the student source package as checked on 7 November 2026. Source code and `--help` are the final reference for exact command behavior.